BIRZEIT UNIVERSITY

Faculty of Engineering & Technology

Electrical & Computer Engineering Department

ADVANCED DIGITAL SYSTEMS DESIGN

-ENCS3310-

Course Project

2-digit BCD adder (8 bits)

Prepared by: Ayah Saad

ID: 1191334

Instructor's Name: Dr. Abdallatif Abuissa

Section: 1

Date: 11-5-2022

➢ Brief introduction and background

The aim of this project is to design a 2-digit BCD adder (8 bits) structurally, from basic logic gates with specific delays, and then blocks of the BCD adder itself assembled from these gates, after that built a complete code for functional verification in order to test out the results of all possible values entering the BCD adder. Binary-Coded Decimal Code BCD is a class of binary encodings of decimal numbers where each decimal digit is represented by a fixed number of bits, four bits. A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9. A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's. [2]

Table of Contents

- Brief introduction and background 1
- Design philosophy 4
  1. Stage 1: (Ripple carry adder) 5
  - One-Bit Full Adder 6
  - Four-Bit Full Adder 6
  - 1-digit BCD adder 6
  - 2-digit BCD adder 7
  - All System: 8
  2. Stage 2 10
  - Four-Bit Carry look ahead Adder 11
- Results 12

  Stage 1: (Ripple carry adder) 12

  Stage 2: (Carry Look Ahead) 14
- Conclusion and Future works 15

References 16

List of Figuers

Figure 1: BCD Adder Logic Circuit 4
Figure 2: logic diagram of a 4-bit ripple carry adder 5
Figure 3: 1-bit full adder 6
Figure 4: 4-bit full adder 6
Figure 5: 1-digit BCD adder 7
Figure 6: 2-digit BCD adder 7
Figure 7: All System 8
Figure 8: D-Flip flop 8
Figure 9: 16-bit Register 9
Figure 10: 9-bit Register 9
Figure 11: 4-bit-Carry-Look-ahead-Adder-Logic-Diagram 10
Figure 12: 1-Bit Carry look ahead Adder 11
Figure 13: 4-bit carry look ahead adder 11
Figure 14: One-Bit Full Adder 12
Figure 15: 4-bit full adder 12
Figure 16: BCD ripple adder 12
Figure 17: 2-Digit BCD Adder) 13
Figure 18: All-System 13
Figure 19: All-System 13
Figure 20: Carry look ahead adder 14
Figure 21: Carry look ahead adder 14

➢ Design philosophy

- **Gates:**

  Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on certain **logic**. [3]

  First step in this project was to design the gates structurally with the given delays.

Gate	Delay
Inverter	3 ns
NAND	6 ns
NOR	6 ns
AND	8 ns
OR	8 ns
XNOR	10 ns
XOR	12 ns

➢ BCD Adder

Figure 1 illustrates three parts of a BCD adder: 4-bit binary adder, over 9 detection unit and correction unit. The first part is a binary adder which performs on two four-bit BCD digits and a one-bit carry input. In the second part, the over-9-detector recognizes if the result of the first part is more than 9 or not. Finally, in the third part, if the output of detector is '1', the sum is added by 6, else do nothing. The 4-bit binary adder is cascade of 4 FAs (4-bit carry-propagate adder). The detection part is constructed by using two AND gates and one OR gate. The correction unit adds ‘0’ to the binary number if the binary result is less than 10 and adds 6 to the binary result if it is more than 9. [4]

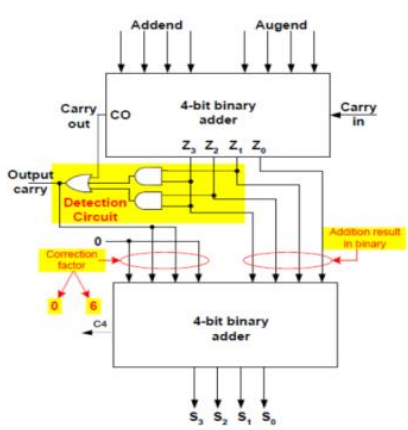


Figure 1: BCD Adder Logic Circuit

BCD adder was split into two stages of complexity: First by using the ripple carry adders, and second by using the carry look ahead.

1. Stage 1: (Ripple carry adder)

Multiple full adder circuits can be cascaded in parallel to add an N-bit number. For an N-bit parallel adder, there must be N number of full adder circuits.

A ripple carry adder is a logic circuit in which the carry-out of each full adder is the carry in of the succeeding next most significant full adder. It is called a ripple carry adder because each carry bit gets rippled into the next stage. In a ripple carry adder the sum and carry out bits of any half adder stage is not valid until the carry in of that stage occurs. Propagation delays inside the logic circuitry are the reason behind this. Propagation delay is time elapsed between the application of an input and occurrence of the corresponding output. Consider a NOT gate, When the input is "0" the output will be "1" and vice versa. The time taken for the NOT gate's output to become "0" after the application of logic "1" to the NOT gate's input is the propagation delay here. Similarly the carry propagation delay is the time elapsed between the application of the carry in signal and the occurrence of the carry out (Cout) signal. Circuit diagram of a 4-bit ripple carry adder is shown below.[5]

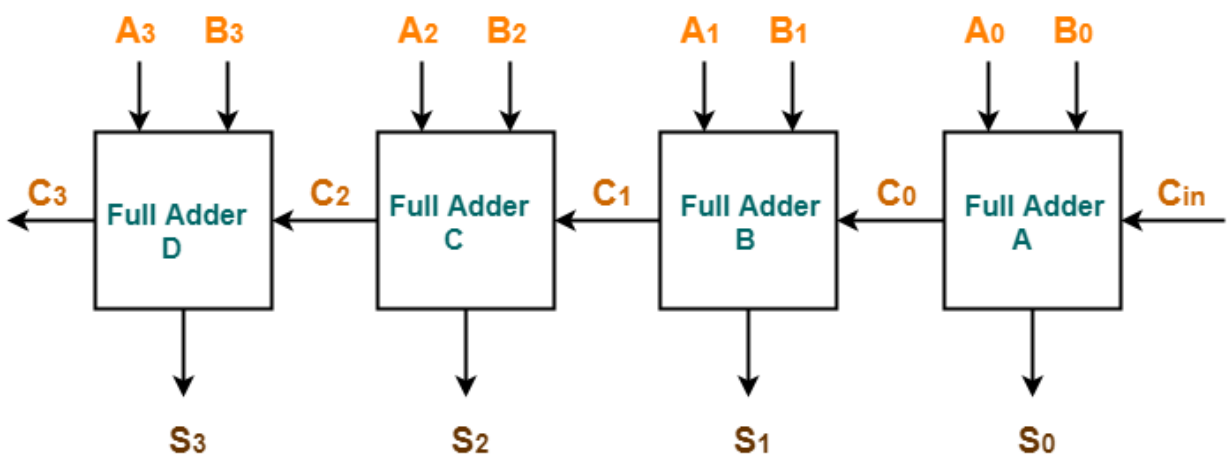


Figure 2: logic diagram of a 4-bit ripple carry adder

➢ One-Bit Full Adder

Here I start from the basic unit which is 1 -bit full adder; it has 3 inputs (x, y, cin) and 2 outputs (sum, cout). This full adder is implemented using 2 XORs, 2 ANDs and 1 OR gate

```
//Structural code for 1-bit full adder

module One_Bit_Full_Adder(input X1, X2, Cin, output Sum, Cout);

 wire a1, a2, a3;

 //a1=x1^x2
 //a2=x1&x2
 //a3=x1&x2&cin

 xor #12ns u1(a1,X1,X2);
 and #8ns u2(a2,X1,X2);
 and #8ns u3(a3,a1,Cin);
 or  #8ns u4(Cout,a2,a3);
 xor #12ns u5(Sum,a1,Cin);

endmodule
```

Figure 3: 1-bit full adder

➢ Four-Bit Full Adder

In this step I combine 4 1 -bit full adders to create the 4-bit full adder. In this configuration it has 4bits per each input with a carry in has 4bit output with a carry out.

```
//Structural module for a 4-bit ripple adder

module ripple_adder_4 (x,y,sum,cout,cin);

    input [0:3]x,y; //two 4-bit inputs
    input cin;

    output [0:3] sum;
    output cout;

    wire n1,n2,n3;

    // instantiating 4 1-bit full adders in Verilog

    One_Bit_Full_Adder FA1(x[3],y[3],cin,sum[3],n1);
    One_Bit_Full_Adder FA2(x[2],y[2],n1,sum[2],n2);
    One_Bit_Full_Adder FA3(x[1],y[1],n2,sum[1],n3);
    One_Bit_Full_Adder FA4(x[0],y[0],n3,sum[0],cout);

endmodule
```

Figure 4: 4-bit full adder

➢ 1-digit BCD adder

Here arithmetic operations are done using BCD number format. Hence the result of normal binary addition should convert to BCD format using error correction methods. For this configuration I use two 4-bit binary adders, 2 AND gates and 1 OR gate. In the error correction, 0110 binary 6 should add to the result of normal binary addition to convert the result in BCD format.

```
//Structural module for a BCD adder

module Bcd_adder(x,y,cin,c_out,sum);
    //inputs
    input [0:3]x,y;
    input cin;
    //outputs
    output c_out;
    output [0:3]sum;
    wire z; // carry out of 4-bit Binary adder #1
    wire n1,n2;
    /*
    n1=wire of and gate #1
    n2=wire of and gate #2
    */
    wire m;                //cout of BCD #2 and was ignored
    wire [0:3] l;          //Erorr detection
    wire [0:3]wire_sum;  //the output sum of 4-bit Binary adder #1
    and (n1,wire_sum[0],wire_sum[1]);
    and (n2,wire_sum[0],wire_sum[2]);
    or  (c_out,n1,n2,z);

    //four bit adder body from instance of full_adder
    ripple_adder_4 add0(x,y,wire_sum,z,cin);
    assign l= {1'b0,c_out,c_out,1'b0};
    ripple_adder_4 add1(wire_sum,l,sum,m,1'b0);

endmodule
```

Figure 5: 1-digit BCD adder

➢ 2-digit BCD adder

In this step I combine 2 digit BCD adders to create the 2-digit BCD adder.

```
//Build 2-Digit BCD Adder from 1-Digit BCD

module Two_Digit_BCD(x,y,sum,COUT);

    //inputs
    input [0:7] x;
    input [0:7] y;

    //outputs
    output COUT;
    output [0:7]sum;

    wire cout;

     Bcd_adder b1(x[4:7],y[4:7],1'b0,cout,sum[4:7]);
     Bcd_adder b2(y[0:3],y[0:3],cout,COUT,sum[0:3]);
endmodule
```

Figure 6: 2-digit BCD adder

➢ All System:

In this step I combine 2 digit BCD adders with 16-bit Register for input and 9-bit Register for the output to create the 2-digit BCD adder with 2-Registers.

```
//Structural module for all the system

module All_System(x,y,cin,sum,CLK,RST);

    //inputs
    input [0:7] x,y;
    input cin,CLK,RST;
    //outputs
    output [0:8]sum;
    wire cout;
    wire c_out;
    wire [0:15]SUM;
    wire [0:7]out;

    Sixteen_Bit_Register inst1({x,y},CLK,RST,SUM);      //input Register
    Nine_Bit_Register inst2({c_out,out},CLK,RST,sum);   //output Register

    Bcd_adder u1(SUM[4:7],SUM[12:15],1'b0,cout,out[4:7]);
    Bcd_adder u2(SUM[0:3],SUM[8:11],cout,c_out,out[0:3]);

endmodule
```

Figure 7: All System

```
//D flip-flop with asynchronous reset.

module DFF (D,CLK,Q,Qb,RST);
   output Q,Qb;
   input D,CLK,RST;
   reg Q;
   assign Qb = ~ Q ;
   always @(posedge CLK or negedge RST)
       begin
         if (~RST) Q = 1'b0;
         else Q = D;
     end

endmodule
```

Figure 8: D-Flip flop

```
module Sixteen_Bit_Register (a,clk,RST,q);
    parameter N=16;
    input [0:N-1] a;
    input clk,RST;
    output wire [0:N-1] q;
    wire [0:N-1] qb;
    reg [0:N-1] d;

    //declare a loop   variable to be used during Genaration
    genvar i;
    //Generate for loop to instantiate N times
    generate
    for (i=0; i<N; i=i+1)
        begin
            DFF u0(d[i],clk,q[i],qb[i],RST);
        end
        endgenerate

    always @(posedge clk or negedge RST)
        begin
         if (~RST) d = 1'b0;
         else
             d[0]=a[0];d[1]=a[1];d[2]=a[2];d[3]=a[3];d[4]=a[4];d[5]=a[5];d[6]=a[6];d[7]=a[7];
             d[8]=a[8];d[9]=a[9];d[10]=a[10];d[11]=a[11];d[12]=a[12];d[13]=a[13];d[14]=a[14];
             d[15]=a[15];
         end
endmodule
```

Figure 9: 16-bit Register

```
//9-bit Register
module Nine_Bit_Register(d,CLK,RST,q);
    input [8:0]d;
    input CLK,RST;
    output reg [8:0]q;

     always @ (posedge CLK or negedge RST)
       if (!RST)  q <= 0;
       else
          q <= d;

endmodule
```

Figure 10: 9-bit Register

2. Stage 2

Carry Look-ahead Adder is the faster adder circuit. It reduces the propagation delay, which occurs during addition, by using more complex hardware circuitry. It is designed by transforming the ripple-carry Adder circuit such that the carry logic of the adder is changed into two-level logic. [6]

4-Bit Carry Look-ahead Adder

Here the sum S3 can be produced as soon as the inputs A3 and B3 are given. But carry C3 cannot be computed until the carry bit C2 is applied whereas C2 depends on C1. Therefore to produce final steady-state results, carry must propagate through all the states. This increases the carry propagation delay of the circuit.

The propagation delay of the adder is calculated as "the propagation delay of each gate times the number of stages in the circuit". For the computation of a large number of bits, more stages have to be added, which makes the delay much worse. Hence, to solve this situation, Carry Look-ahead Adder was introduced.

To understand the functioning of a Carry Look-ahead Adder, a 4-bit Carry Look-ahead Adder is described below.

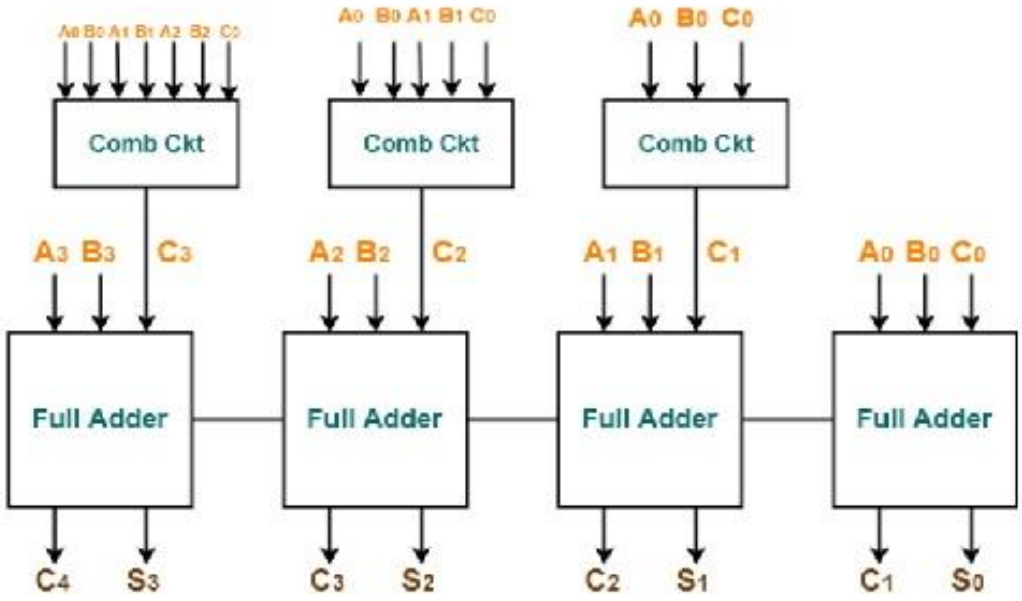


Figure 11: 4-bit-Carry-Look-ahead-Adder-Logic-Diagram

➢ One-Bit Full Adder

Here I start from the basic unit which is 1 -bit carry ahead adder. This adder is implemented using 2 XORs, 1 ANDs and 1 OR gate

```
////////////////////////// STAGE 2 ////////////////////////////

// 1-bit Carry Look ahead module

module Look_Ahead (x,y,cin,sum,a,b);

     //inputs
     input x,y,cin;
     output sum,a,b;
     wire w;

     //outputs
     xor #12ns u1(w,x,y);
     and #8ns u3(b,x,y);
     or  #8ns u2(a,x,y);
     xor #12ns u4(sum,w,cin);

endmodule
```

Figure 12: 1-Bit Carry look ahead Adder

➢ Four-Bit Carry look ahead Adder

In this step I combine 4 1 -bit carry look ahead adders to create the 4-bit adder.

```
// 4-bit Carry Look ahead module

module Carry_Lookahead_4_Bit (A,B,S,Cout,Cin);

    //inputs
    input [0:3]A,B;
    input Cin;

    //outputs
    output Cout;
    output [0:3]S;
    wire [0:21]w;
    //instantiate the module
    Look_Ahead U1 (A[3],B[3],Cin,S[3],w[21],w[20]);
    or #8ns (w[18],w[19],w[20]);
    and #8ns (w[19],Cin,w[21]);

    //instantiate the module
    Look_Ahead U2(A[2],B[2],w[18],S[2],w[17],w[16]);
    or #8ns (w[13],w[16],w[14],w[15]);
    and #8ns (w[14],w[20],w[17]);
    and #8ns (w[15],Cin,w[21],w[17]);

    //instantiate the module
    Look_Ahead U3(A[1],B[1],w[13],S[1],w[12],w[11]);
    or #8ns (w[7],w[11],w[10],w[9],w[8]);
    and #8ns (w[8],w[16],w[12]);
    and #8ns  (w[9],w[20],w[17],w[12]);
    and #8ns (w[10],cin,w[21],w[17],w[12]);
```

Figure 13: 4-bit carry look ahead adder

```
    //instantiate the module
    Look_Ahead U1 (A[3],B[3],Cin,S[3],w[21],w[20]);
    or #8ns (w[18],w[19],w[20]);
    and #8ns (w[19],Cin,w[21]);

    //instantiate the module
    Look_Ahead U2(A[2],B[2],w[18],S[2],w[17],w[16]);
    or #8ns (w[13],w[16],w[14],w[15]);
    and #8ns (w[14],w[20],w[17]);
    and #8ns (w[15],Cin,w[21],w[17]);

    //instantiate the module
    Look_Ahead U3(A[1],B[1],w[13],S[1],w[12],w[11]);
    or #8ns (w[7],w[11],w[10],w[9],w[8]);
    and #8ns (w[8],w[16],w[12]);
    and #8ns  (w[9],w[20],w[17],w[12]);
    and #8ns (w[10],cin,w[21],w[17],w[12]);

    //instantiate the module
    Look_Ahead U4(A[0],B[0],w[7],S[0],w[6],w[5]);
    or #8ns (w[0],w[1],w[2],w[3],w[4]);
    and #8ns (w[1],w[11],w[6]);
    and #8ns (w[2],w[16],w[12],w[6]);
    and #8ns (w[3],w[20],w[17],w[12],w[6]);
    and #8ns (w[4],Cin,w[21],w[17],w[12],w[6]);
    assign Cout = w[0];

endmodule
```

➢ Results

Stage 1: (Ripple carry adder)

➢ One-Bit Full Adder

Testbench was made to check for the output one each step.

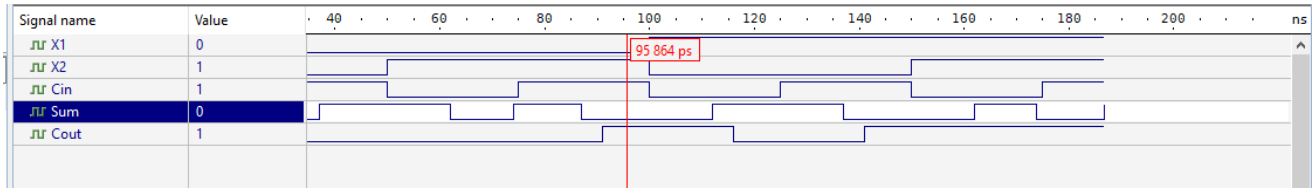


Figure 14: One-Bit Full Adder

➢ Four-Bit Full Adder

Testbench was made to check for the output one each step.

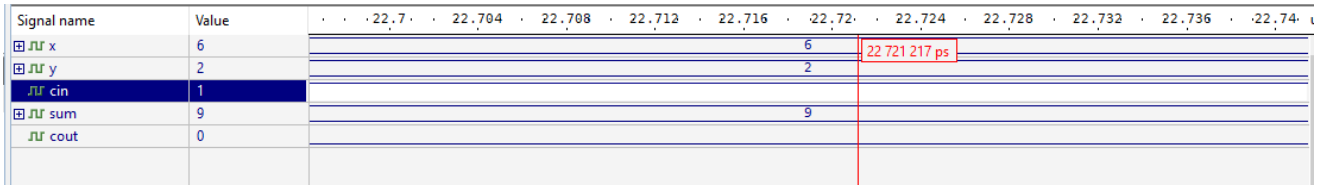


Figure 15: 4-bit full adder

➢ 1-digit BCD Adder

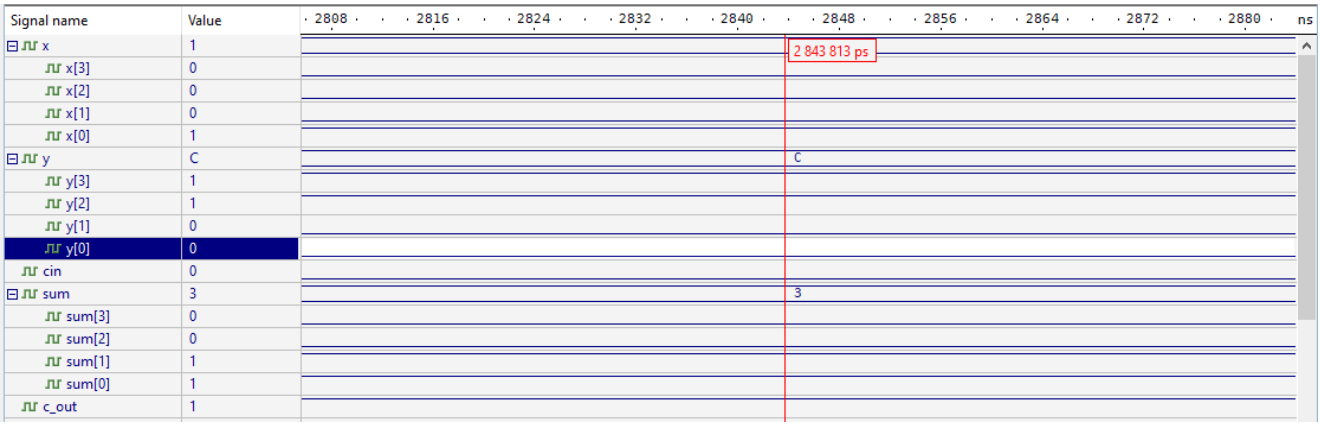


Figure 16: BCD ripple adder

➢ (2-Digit BCD Adder)

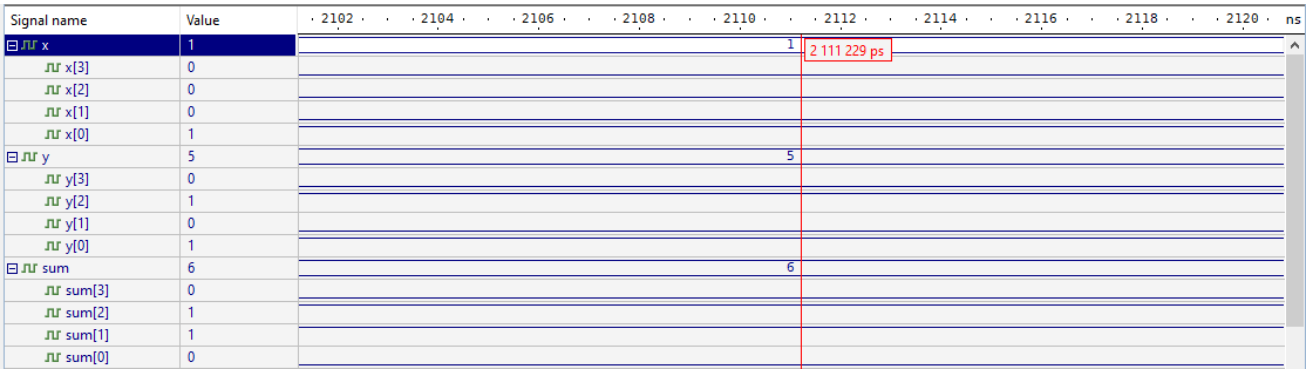


Figure 17: 2-Digit BCD Adder)

➢ All-System

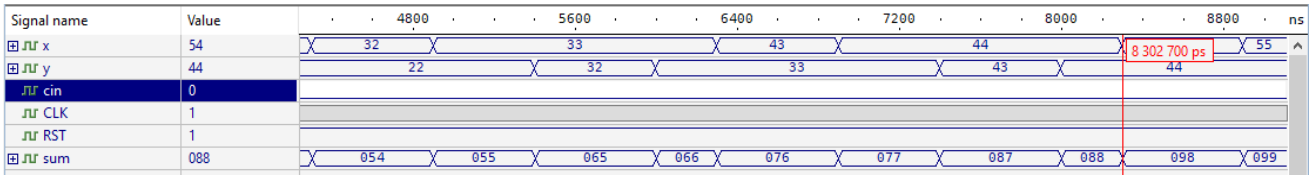


Figure 18: All-System

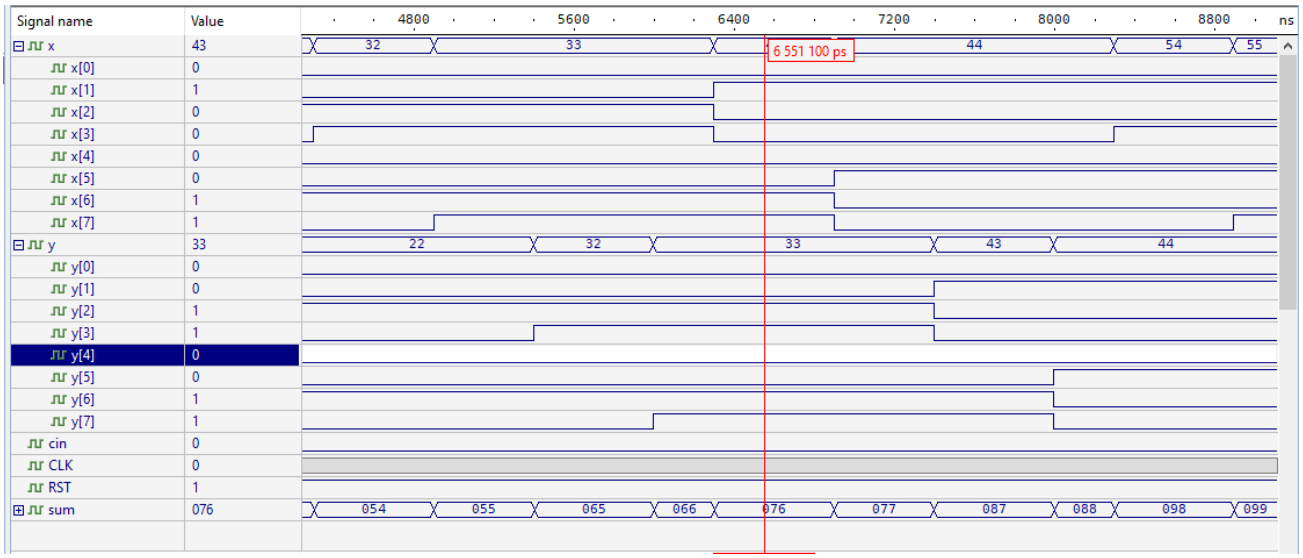
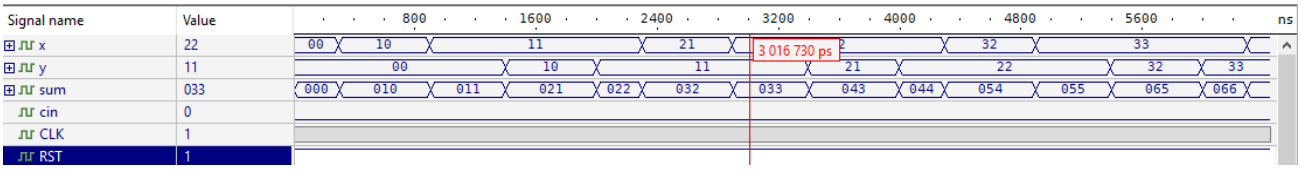


Figure 19: All-System



Stage 2: (Carry Look Ahead)

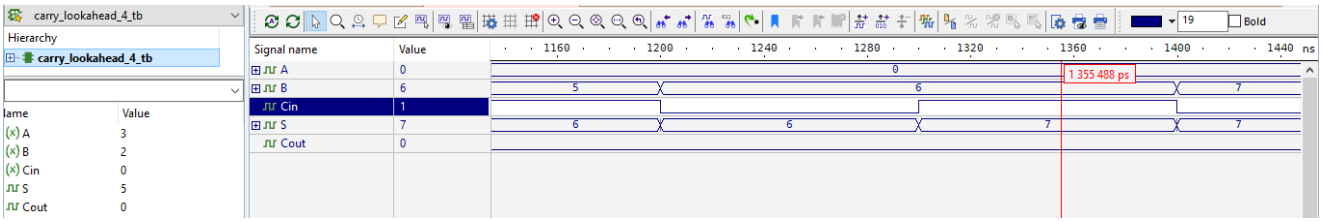


Figure 20: Carry look ahead adder

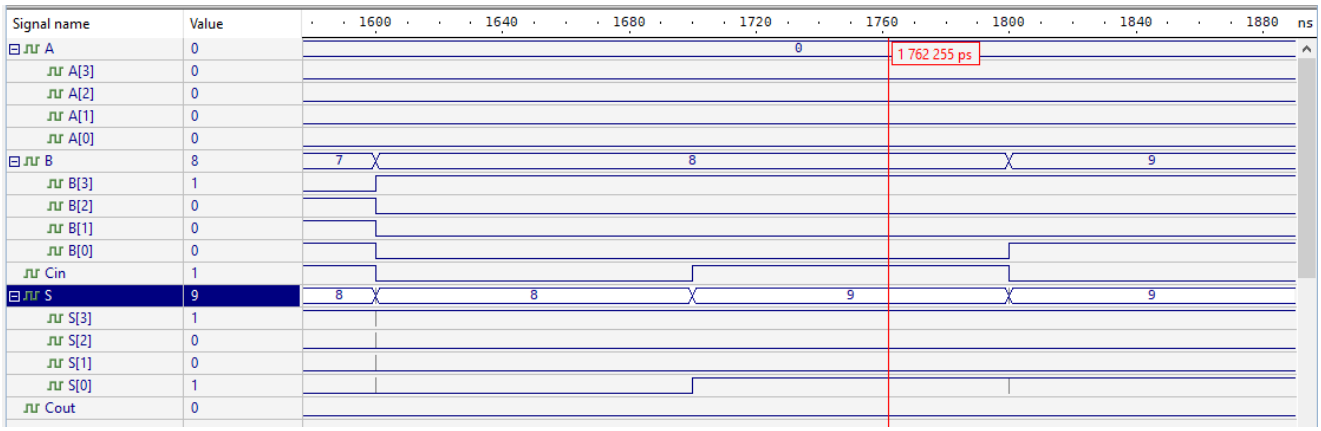


Figure 21: Carry look ahead adder

➢ Conclusion and Future works

In this project we used different delay values depending on the type of gate we used.

Carry look ahead adders are similar to Ripple Carry Adders. The difference is that carry lookahead adders are able to calculate the Carry bit before the Full Adder is done with its operation. This gives it an advantage over the Ripple Carry Adder because it is able to add two numbers together faster. The drawback is that it takes more logic.

We became more familiar with Aldec active HDL, to simulate our project and see the wave form for module that we worked on it., also we learned how to print an error, make a delay, testing the systems and building structural module.

References

[1] Advanced Digital System Design slides of Dr. Abdallatif Abuissa

[2] https://site.ucas.edu.ps/Portals/1770/Lab-5-BCD-Adder-and-Comparator.pdf

Accessed on 6/5/2022 at 3:26 PM
[3] https://www.tutorialspoint.com/computer_logical_organization/logic_gates.htm
Accessed on 6/5/2022 at 3:37 PM
[4] https://www.idosi.org/wasj/wasj4(6)/5.pdf

Accessed on 6/5/2022 at 4:05 PM
[5] https://www.circuitstoday.com/ripple-carry-adder

Accessed on 6/5/2022 at 4:27 PM
[6] https://www.elprocus.com/carry-look-ahead-adder/
Accessed on 6/5/2022 at 4:40 PM